CROP DISEASE DETECTION APP

Detailed Build Prompt Library

PHASE 3 — Frontend (React)

Prompts 111–165 · 55 Fully Detailed Prompts

Prepared for: Priyanshu Kumar Gupta & Team
(Sadhana Bag, Kumar Gaurav, Rajdeep Kumar)

Part 3 of 4 — execute this phase fully before opening the next

Phase Overview

This phase builds the multi-user interface: authentication screens, the personalized dashboard, the scan flow, and every screen that makes the app feel like each user's own.

Reference: Full Project File Structure

Every prompt in this document references paths from the following master file tree. Keep it visible while executing.

```
crop-disease-detection/
|-- README.md
|-- ml/
|   |-- requirements.txt
|   |-- dataset/
|   |   |-- raw/
|   |   |-- filtered/
|   |   |-- split/
|   |   |   |-- train/<class>/
|   |   |   |-- val/<class>/
|   |   |   `-- test/<class>/
|   |   `-- real_world_sample/<class>/
|   |-- notebooks/
|   |   |-- 00_colab_setup.ipynb
|   |   |-- 01_dataset_exploration.ipynb
|   |   `-- 02_preprocessing_test.ipynb
|   |-- preprocessing/
|   |   |-- loader.py
|   |   |-- augmentation.py
|   |   |-- dataset_pipeline.py
|   |   `-- class_weights.py
|   |-- training/
|   |   |-- baseline_cnn.py
|   |   |-- train_baseline.py
|   |   |-- transfer_mobilenetv2.py
|   |   |-- train_transfer.py
|   |   `-- transfer_resnet50.py
|   |-- evaluation/
|   |   |-- evaluate.py
|   |   |-- confusion_matrix.py
|   |   |-- compare_models.py
|   |   |-- confidence_threshold_experiment.py
|   |   `-- real_world_eval.py
```

```
|   `-- models/
|       |-- crop_disease_model.keras
|       `-- class_names.json
|
|-- backend/
|   |-- requirements.txt
|   |-- .env.example
|   |-- alembic/
|   |-- scripts/
|   |   `-- reset_db.py
|   |-- tests/
|   |   |-- conftest.py
|   |   |-- test_auth.py
|   |   |-- test_users.py
|   |   |-- test_predict.py
|   |   `-- test_history_feedback.py
|   `-- app/
|       |-- main.py
|       |-- core/
|       |   |-- config.py
|       |   |-- security.py
|       |   `-- dependencies.py
|       |-- db/
|       |   |-- session.py
|       |   `-- seed_diseases.py
|       |-- models/
|       |   |-- user.py
|       |   |-- prediction.py
|       |   |-- disease.py
|       |   `-- feedback.py
|       |-- schemas/
|       |   |-- auth.py
|       |   |-- user.py
|       |   `-- prediction.py
|       |-- routes/
|       |   |-- auth.py
|       |   |-- users.py
|       |   |-- predict.py
|       |   `-- diseases.py
|       |-- services/
|       |   |-- model_service.py
|       |   `-- preprocessing_service.py
|       `-- disease_data/
|
|-- frontend/
|   |-- package.json
|   |-- tailwind.config.js
|   |-- index.html
|   `-- src/
|       |-- App.tsx
|       |-- components/
|       |   |-- ui/
|       |   |   |-- Button.tsx
|       |   |   |-- Card.tsx
|       |   |   |-- Input.tsx
|       |   |   |-- Avatar.tsx
|       |   |   `-- Spinner.tsx
|       |   |-- StatsCard.tsx
|       |   |-- RecentScansTable.tsx
|       |   |-- ProtectedRoute.tsx
|       |   `-- Layout.tsx
|       |-- pages/
|       |   |-- Signup.tsx
|       |   |-- Login.tsx
|       |   |-- Dashboard.tsx
|       |   |-- NewScan.tsx
|       |   |-- Analyzing.tsx
|       |   |-- Result.tsx
```

```
|       |   |-- History.tsx
|       |   |-- Profile.tsx
|       |   `-- About.tsx
|       |-- services/
|       |   `-- api.ts
|       |-- store/
|       |   `-- authStore.ts
|       |-- hooks/
|       `-- assets/
|
`-- docs/
    |-- SETUP.md
    |-- ARCHITECTURE.md
    |-- API_REFERENCE.md
    |-- DATABASE_SCHEMA.md
    |-- SECURITY.md
    |-- PERFORMANCE.md
    |-- TESTING_REPORT.md
    `-- DEMO_SCRIPT.md
```

Detailed Prompts

3.A Project Setup & Design System

PROMPT 111 — Initialize frontend/ as a React project (Vite) with TypeScript, and install Tailwind CSS, ...

Objective: Initialize `frontend/` as a React project (Vite) with TypeScript, and install Tailwind CSS, React Router, and Zustand.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): `frontend/`

Detailed Requirements

- Initialize `frontend/` as a React project (Vite) with TypeScript, and install Tailwind CSS, React Router, and Zustand.
- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.
- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

PROMPT 112 — Create the folder structure

Objective: Create the folder structure: `frontend/src/{components/, pages/, services/, store/, hooks/, assets/}`.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): `frontend/src/{components/, pages/, services/, store/, hooks/, assets/}`

Detailed Requirements

- Create the folder structure: `frontend/src/{components/, pages/, services/, store/, hooks/, assets/}`.

- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.
- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

PROMPT 113 — Configure Tailwind with the project's design tokens

Objective: Configure Tailwind with the project's design tokens: primary green palette, neutral greys, spacing scale, and typography.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Configure Tailwind with the project's design tokens: primary green palette, neutral greys, spacing scale, and typography.
- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.

- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

PROMPT 114 — Build a base frontend/src/components/ui/Button.tsx component with primary/secondary/ghost ...

Objective: Build a base `frontend/src/components/ui/Button.tsx` component with primary/secondary/ghost variants.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): `frontend/src/components/ui/Button.tsx`

Detailed Requirements

- Build a base `frontend/src/components/ui/Button.tsx` component with primary/secondary/ghost variants.
- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.
- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

PROMPT 115 — Build base Card.tsx, Input.tsx, and Avatar.tsx (with initials-fallback) components in ...

Objective: Build base `Card.tsx`, `Input.tsx`, and `Avatar.tsx` (with initials-fallback) components in `frontend/src/components/ui/`.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): `Card.tsx`, `Input.tsx`, `Avatar.tsx`, `frontend/src/components/ui/`

Detailed Requirements

- Build base `Card.tsx`, `Input.tsx`, and `Avatar.tsx` (with initials-fallback) components in `frontend/src/components/ui/`.
- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.
- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

PROMPT 116 — Build a Spinner.tsx loading-state component for the "Analyzing

Objective: Build a `Spinner.tsx` loading-state component for the "Analyzing…" screen.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): `Spinner.tsx`

Detailed Requirements

- Build a `Spinner.tsx` loading-state component for the "Analyzing…" screen.
- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.
- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

PROMPT 117 — Set up React Router in frontend/src/App.tsx with routes for /login, /signup, /dashboa...

Objective: Set up React Router in `frontend/src/App.tsx` with routes for `/login`, `/signup`, `/dashboard`, `/scan`, `/history`, `/profile`, `/about`.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): `frontend/src/App.tsx`, `/login`, `/signup`, `/dashboard`, `/scan`, `/history`, `/profile`, `/about`

Detailed Requirements

- Set up React Router in `frontend/src/App.tsx` with routes for `/login`, `/signup`, `/dashboard`, `/scan`, `/history`, `/profile`, `/about`.
- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.
- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

PROMPT 118 — Create frontend/src/services/api.ts with a configured fetch wrapper that attaches the JWT ...

Objective: Create `frontend/src/services/api.ts` with a configured fetch wrapper that attaches the JWT Authorization header automatically.

Why This Matters: This foundational frontend work defines the visual language and base components every page in Phase 3 will reuse. Getting the design tokens and base components right here means every later page prompt can focus purely on layout/logic, not re-deriving styling.

Primary File(s): `frontend/src/services/api.ts`

Detailed Requirements

- Create `frontend/src/services/api.ts` with a configured fetch wrapper that attaches the JWT Authorization header automatically.
- Define design tokens (colors, spacing, typography) centrally in the Tailwind config, not as one-off inline values scattered across components.
- Base UI components should accept standard props (e.g. `variant`, `className`, `children`) so they compose cleanly across pages.
- Keep components presentational (no direct API calls) — data-fetching belongs in page components or hooks, not base UI primitives.
- Export every new component from a clear, predictable path so later prompts can import it without guessing.

Technical Specification

Stack: React + TypeScript + Vite + Tailwind CSS + React Router + Zustand. Component files live under `frontend/src/components/ui/`.

Acceptance Criteria

- ☐ Component renders correctly in isolation with representative sample props.
- ☐ No hard-coded colors/spacing outside the Tailwind config/theme.
- ☐ TypeScript compiles with no type errors.

Edge Cases & Notes

- Confirm the component behaves sensibly with missing/empty optional props (e.g. `Avatar` with no photo URL falls back to initials, not a broken image icon).

3.B Auth State & Pages

PROMPT 119 — Create frontend/src/store/authStore.ts (Zustand) holding the current user, JWT token, and ...

Objective: Create `frontend/src/store/authStore.ts` (Zustand) holding the current user, JWT token, and login/logout actions, persisted to localStorage.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/store/authStore.ts`

Detailed Requirements

- Create `frontend/src/store/authStore.ts` (Zustand) holding the current user, JWT token, and login/logout actions, persisted to localStorage.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 120 — Build frontend/src/pages/Signup.tsx with a form for name, email, password, optional phone/...

Objective: Build `frontend/src/pages/Signup.tsx` with a form for name, email, password, optional phone/location, calling `POST /auth/register`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/pages/Signup.tsx`, `POST /auth/register`

Detailed Requirements

- Build `frontend/src/pages/Signup.tsx` with a form for name, email, password, optional phone/location, calling `POST /auth/register`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 121 — Build frontend/src/pages/Login.tsx with a form for email/password, calling POST /auth/login

Objective: Build `frontend/src/pages/Login.tsx` with a form for email/password, calling `POST /auth/login`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/pages/Login.tsx`, `POST /auth/login`

Detailed Requirements

- Build `frontend/src/pages/Login.tsx` with a form for email/password, calling `POST /auth/login`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).

- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 122 — Add client-side form validation (required fields, email format, password length) to both aut...

Objective: Add client-side form validation (required fields, email format, password length) to both auth forms.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add client-side form validation (required fields, email format, password length) to both auth forms.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 123 — Add error-message display for failed login/signup attempts (wrong password, duplicate email)

Objective: Add error-message display for failed login/signup attempts (wrong password, duplicate email).

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add error-message display for failed login/signup attempts (wrong password, duplicate email).
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 124 — Create a ProtectedRoute.tsx wrapper component that redirects unauthenticated users to /lo...

Objective: Create a `ProtectedRoute.tsx` wrapper component that redirects unauthenticated users to `/login`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `ProtectedRoute.tsx`, `/login`

Detailed Requirements

- Create a `ProtectedRoute.tsx` wrapper component that redirects unauthenticated users to `/login`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 125 — Wire successful login/signup to store the JWT and redirect to /dashboard

Objective: Wire successful login/signup to store the JWT and redirect to `/dashboard`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `/dashboard`

Detailed Requirements

- Wire successful login/signup to store the JWT and redirect to `/dashboard`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.

- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 126 — Implement logout functionality (clear token, redirect to /login) accessible from the navba...

Objective: Implement logout functionality (clear token, redirect to `/login`) accessible from the navbar dropdown.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `/login`

Detailed Requirements

- Implement logout functionality (clear token, redirect to `/login`) accessible from the navbar dropdown.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).

- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

3.C Personalized Dashboard

PROMPT 127 — Build frontend/src/pages/Dashboard.tsx with a greeting header ("Welcome back, {user

Objective: Build `frontend/src/pages/Dashboard.tsx` with a greeting header ("Welcome back, {user.name}!") using the authenticated user's data.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/pages/Dashboard.tsx`

Detailed Requirements

- Build `frontend/src/pages/Dashboard.tsx` with a greeting header ("Welcome back, {user.name}!") using the authenticated user's data.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 128 — Fetch and display the user's avatar/photo in the dashboard header, falling back to initials ...

Objective: Fetch and display the user's avatar/photo in the dashboard header, falling back to initials if no photo is set.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Fetch and display the user's avatar/photo in the dashboard header, falling back to initials if no photo is set.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 129 — Build frontend/src/components/StatsCard.tsx and wire three cards (Total Scans, Diseases De...

Objective: Build `frontend/src/components/StatsCard.tsx` and wire three cards (Total Scans, Diseases Detected, Healthy Confirmed) to `GET /users/me/stats`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/components/StatsCard.tsx`, `GET /users/me/stats`

Detailed Requirements

- Build `frontend/src/components/StatsCard.tsx` and wire three cards (Total Scans, Diseases Detected, Healthy Confirmed) to `GET /users/me/stats`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.

- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 130 — Build frontend/src/components/RecentScansTable.tsx showing the 5 most recent predictions, ...

Objective: Build `frontend/src/components/RecentScansTable.tsx` showing the 5 most recent predictions, fetched from `GET /predictions/history`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/components/RecentScansTable.tsx`, `GET /predictions/history`

Detailed Requirements

- Build `frontend/src/components/RecentScansTable.tsx` showing the 5 most recent predictions, fetched from `GET /predictions/history`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 131 — Add a prominent "+ New Scan" call-to-action button linking to /scan

Objective: Add a prominent "+ New Scan" call-to-action button linking to `/scan`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `/scan`

Detailed Requirements

- Add a prominent "+ New Scan" call-to-action button linking to `/scan`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 132 — Add loading and empty states to the dashboard (e

Objective: Add loading and empty states to the dashboard (e.g. "No scans yet — start your first scan").

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add loading and empty states to the dashboard (e.g. "No scans yet — start your first scan").
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 133 — Add click-through from a recent-scan row to a detailed result view

Objective: Add click-through from a recent-scan row to a detailed result view.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add click-through from a recent-scan row to a detailed result view.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.

- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 134 — Make the dashboard layout responsive for mobile screens (stacked cards, scrollable table)

Objective: Make the dashboard layout responsive for mobile screens (stacked cards, scrollable table).

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Make the dashboard layout responsive for mobile screens (stacked cards, scrollable table).
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.

- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

3.D Scan / Upload Flow

PROMPT 135 — Build frontend/src/pages/NewScan.tsx with a drag-and-drop / click-to-upload image picker

Objective: Build `frontend/src/pages/NewScan.tsx` with a drag-and-drop / click-to-upload image picker.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/pages/NewScan.tsx`

Detailed Requirements

- Build `frontend/src/pages/NewScan.tsx` with a drag-and-drop / click-to-upload image picker.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 136 — Add a client-side image preview before submission

Objective: Add a client-side image preview before submission.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add a client-side image preview before submission.

- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 137 — Add client-side file-type and file-size validation before upload

Objective: Add client-side file-type and file-size validation before upload.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add client-side file-type and file-size validation before upload.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 138 — Wire the upload form to POST /predict with multipart/form-data and the JWT header

Objective: Wire the upload form to `POST /predict` with multipart/form-data and the JWT header.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `POST /predict`

Detailed Requirements

- Wire the upload form to `POST /predict` with multipart/form-data and the JWT header.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 139 — Build an Analyzing.tsx loading state shown while the request is in flight

Objective: Build an `Analyzing.tsx` loading state shown while the request is in flight.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `Analyzing.tsx`

Detailed Requirements

- Build an `Analyzing.tsx` loading state shown while the request is in flight.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 140 — Handle and display network/server errors gracefully on the scan page

Objective: Handle and display network/server errors gracefully on the scan page.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Handle and display network/server errors gracefully on the scan page.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.

- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 141 — Add a placeholder "Open Camera" button (disabled/v2) with a tooltip noting camera capture is...

Objective: Add a placeholder "Open Camera" button (disabled/v2) with a tooltip noting camera capture is a planned enhancement.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add a placeholder "Open Camera" button (disabled/v2) with a tooltip noting camera capture is a planned enhancement.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 142 — Redirect to the Result page automatically on a successful prediction response

Objective: Redirect to the Result page automatically on a successful prediction response.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Redirect to the Result page automatically on a successful prediction response.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

3.E Result Display

PROMPT 143 — Build frontend/src/pages/Result.tsx showing the uploaded image thumbnail, predicted crop, ...

Objective: Build `frontend/src/pages/Result.tsx` showing the uploaded image thumbnail, predicted crop, disease name, and confidence percentage.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/pages/Result.tsx`

Detailed Requirements

- Build `frontend/src/pages/Result.tsx` showing the uploaded image thumbnail, predicted crop, disease name, and confidence percentage.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 144 — Build a confidence-bar or badge component that visually reflects the confidence score

Objective: Build a confidence-bar or badge component that visually reflects the confidence score.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Build a confidence-bar or badge component that visually reflects the confidence score.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 145 — Build collapsible/sectioned display for Symptoms, Prevention, and Management content returne...

Objective: Build collapsible/sectioned display for Symptoms, Prevention, and Management content returned by the API.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Build collapsible/sectioned display for Symptoms, Prevention, and Management content returned by the API.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).

- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 146 — Add a distinct "uncertain result" UI state (warning styling + message) for predictions below...

Objective: Add a distinct "uncertain result" UI state (warning styling + message) for predictions below the confidence threshold.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add a distinct "uncertain result" UI state (warning styling + message) for predictions below the confidence threshold.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.

- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 147 — Add a "Scan Another" button returning to /scan

Objective: Add a "Scan Another" button returning to `/scan`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `/scan`

Detailed Requirements

- Add a "Scan Another" button returning to `/scan`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 148 — Add an optional "Was this correct?" feedback control that calls POST /predictions/{id}/feed...

Objective: Add an optional "Was this correct?" feedback control that calls `POST /predictions/{id}/feedback`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `POST /predictions/{id}/feedback`

Detailed Requirements

- Add an optional "Was this correct?" feedback control that calls `POST /predictions/{id}/feedback`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

3.F History Page

PROMPT 149 — Build frontend/src/pages/History.tsx listing all of the current user's past predictions in...

Objective: Build `frontend/src/pages/History.tsx` listing all of the current user's past predictions in a paginated table.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/pages/History.tsx`

Detailed Requirements

- Build `frontend/src/pages/History.tsx` listing all of the current user's past predictions in a paginated table.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 150 — Add filtering by crop and by disease to the history table

Objective: Add filtering by crop and by disease to the history table.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add filtering by crop and by disease to the history table.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 151 — Add sorting by date and by confidence

Objective: Add sorting by date and by confidence.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add sorting by date and by confidence.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 152 — Add a detail modal/page for viewing a single past prediction's full result again

Objective: Add a detail modal/page for viewing a single past prediction's full result again.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add a detail modal/page for viewing a single past prediction's full result again.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.

- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 153 — Add an empty-state and a loading-skeleton state to the history page

Objective: Add an empty-state and a loading-skeleton state to the history page.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add an empty-state and a loading-skeleton state to the history page.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

3.G Profile / Settings Page

PROMPT 154 — Build frontend/src/pages/Profile.tsx displaying the user's name, email, photo, phone, loca...

Objective: Build `frontend/src/pages/Profile.tsx` displaying the user's name, email, photo, phone, location, and preferred crops.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `frontend/src/pages/Profile.tsx`

Detailed Requirements

- Build `frontend/src/pages/Profile.tsx` displaying the user's name, email, photo, phone, location, and preferred crops.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 155 — Add an "Edit Profile" form wired to PUT /users/me

Objective: Add an "Edit Profile" form wired to `PUT /users/me`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `PUT /users/me`

Detailed Requirements

- Add an "Edit Profile" form wired to `PUT /users/me`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 156 — Add a profile-photo upload control wired to POST /users/me/photo

Objective: Add a profile-photo upload control wired to `POST /users/me/photo`.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): `POST /users/me/photo`

Detailed Requirements

- Add a profile-photo upload control wired to `POST /users/me/photo`.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 157 — Display account metadata (member since, last login) read-only on the profile page

Objective: Display account metadata (member since, last login) read-only on the profile page.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Display account metadata (member since, last login) read-only on the profile page.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.

- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 158 — Add form validation and success/error toasts for profile updates

Objective: Add form validation and success/error toasts for profile updates.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add form validation and success/error toasts for profile updates.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

PROMPT 159 — Add a "Preferred Crops" multi-select/tag input for the user to customize

Objective: Add a "Preferred Crops" multi-select/tag input for the user to customize.

Why This Matters: This page is a direct piece of the personalized user experience described in the project report — it must always scope its data to the currently authenticated user and degrade gracefully (loading/empty/error states) rather than assuming the API call always succeeds instantly.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add a "Preferred Crops" multi-select/tag input for the user to customize.
- Fetch data via the shared `api.ts` client so the JWT Authorization header is attached automatically.
- Implement all three data states explicitly: loading, empty/no-data, and error — never leave the UI blank while waiting.
- Read the current user (name, avatar) from the `authStore` rather than re-fetching it separately on every page.
- Keep the page responsive down to mobile width (this will also be covered again in the 3.I polish pass, but should not be broken from the start).
- Navigate using React Router (`useNavigate` / `<Link>`), never a raw `window.location` redirect.

Technical Specification

Page lives under `frontend/src/pages/`. Pulls data from the corresponding backend endpoint documented in the API Reference (e.g. `/users/me`, `/predictions/history`, `/predict`). Uses Zustand `authStore` for the current user and JWT.

Acceptance Criteria

- ☐ Page correctly reflects the logged-in user's own data only (verified by switching between two test accounts).
- ☐ Loading state is visibly shown during the API request, not a blank screen.
- ☐ An API error is caught and shown as a readable message, not an unhandled exception in the console.

Edge Cases & Notes

- Handle the brand-new-user case (zero scans, no photo, default profile) without layout breakage.
- Handle a slow/failed network request without leaving the page stuck on a spinner forever (add a timeout or retry affordance).

3.H Navigation, Layout & About Page

PROMPT 160 — Build a persistent top navigation bar with logo, nav links, and a user-avatar dropdown (Prof...

Objective: Build a persistent top navigation bar with logo, nav links, and a user-avatar dropdown (Profile / Logout) shown on every authenticated page.

Why This Matters: Shared layout/navigation components appear on every authenticated screen, so correctness and consistency here directly shapes how “personalized and polished” the whole app feels, per the project's core requirement that each user's own identity is always visible.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Build a persistent top navigation bar with logo, nav links, and a user-avatar dropdown (Profile / Logout) shown on every authenticated page.
- Navbar must read the current user's name/avatar from `authStore` and update immediately after login/profile changes — no stale cached values.
- Logout must clear all client-side auth state (token, user object) and redirect to `/login`.
- Layout wrapper should be reused by every authenticated route rather than each page re-implementing its own header/nav.
- Keep navigation accessible (proper link semantics, visible focus states).

Technical Specification

Component lives under `frontend/src/components/`. Reads from `authStore` (Zustand) for identity; uses React Router `<Outlet />` or children composition for the wrapped page content.

Acceptance Criteria

- ☐ Navbar shows the correct, current user's name/photo on every authenticated page.
- ☐ Logout immediately reflects in the UI (redirected to `/login`, protected pages no longer accessible).
- ☐ Component renders consistently across at least three different pages that use it.

Edge Cases & Notes

- Handle a user with no profile photo (initials-avatar fallback) and a very long name (truncate gracefully) without breaking the navbar layout.

PROMPT 161 — Build frontend/src/pages/About.tsx describing the dataset, model, scope, and limitations f...

Objective: Build `frontend/src/pages/About.tsx` describing the dataset, model, scope, and limitations for transparency.

Why This Matters: Shared layout/navigation components appear on every authenticated screen, so correctness and consistency here directly shapes how “personalized and polished” the whole app feels, per the project's core requirement that each user's own identity is always visible.

Primary File(s): `frontend/src/pages/About.tsx`

Detailed Requirements

- Build `frontend/src/pages/About.tsx` describing the dataset, model, scope, and limitations for transparency.

- Navbar must read the current user's name/avatar from `authStore` and update immediately after login/profile changes — no stale cached values.
- Logout must clear all client-side auth state (token, user object) and redirect to `/login`.
- Layout wrapper should be reused by every authenticated route rather than each page re-implementing its own header/nav.
- Keep navigation accessible (proper link semantics, visible focus states).

Technical Specification

Component lives under `frontend/src/components/`. Reads from `authStore` (Zustand) for identity; uses React Router `<Outlet />` or children composition for the wrapped page content.

Acceptance Criteria

- ☐ Navbar shows the correct, current user's name/photo on every authenticated page.
- ☐ Logout immediately reflects in the UI (redirected to `/login`, protected pages no longer accessible).
- ☐ Component renders consistently across at least three different pages that use it.

Edge Cases & Notes

- Handle a user with no profile photo (initials-avatar fallback) and a very long name (truncate gracefully) without breaking the navbar layout.

PROMPT 162 — Build a shared Layout.tsx wrapper combining the navbar and page content consistently acros...

Objective: Build a shared `Layout.tsx` wrapper combining the navbar and page content consistently across all authenticated routes.

Why This Matters: Shared layout/navigation components appear on every authenticated screen, so correctness and consistency here directly shapes how “personalized and polished” the whole app feels, per the project's core requirement that each user's own identity is always visible.

Primary File(s): `Layout.tsx`

Detailed Requirements

- Build a shared `Layout.tsx` wrapper combining the navbar and page content consistently across all authenticated routes.
- Navbar must read the current user's name/avatar from `authStore` and update immediately after login/profile changes — no stale cached values.
- Logout must clear all client-side auth state (token, user object) and redirect to `/login`.
- Layout wrapper should be reused by every authenticated route rather than each page re-implementing its own header/nav.
- Keep navigation accessible (proper link semantics, visible focus states).

Technical Specification

Component lives under `frontend/src/components/`. Reads from `authStore` (Zustand) for identity; uses React Router `<Outlet />` or children composition for the wrapped page content.

Acceptance Criteria

- ☐ Navbar shows the correct, current user's name/photo on every authenticated page.
- ☐ Logout immediately reflects in the UI (redirected to `/login`, protected pages no longer accessible).
- ☐ Component renders consistently across at least three different pages that use it.

Edge Cases & Notes

- Handle a user with no profile photo (initials-avatar fallback) and a very long name (truncate gracefully) without breaking the navbar layout.

3.I Responsive & Visual Polish

PROMPT 163 — Do a full responsive-design pass across all pages at mobile, tablet, and desktop breakpoints

Objective: Do a full responsive-design pass across all pages at mobile, tablet, and desktop breakpoints.

Why This Matters: A final responsive/visual QA pass is what separates a working prototype from something that looks and feels like a real product — directly supporting the project's goal that the app feel personally owned and professionally built, not a rough student demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Do a full responsive-design pass across all pages at mobile, tablet, and desktop breakpoints.
- Test every page at three breakpoints: mobile (~375px), tablet (~768px), and desktop (~1280px+).
- Ensure every data-driven component (stats cards, tables, result sections) has a defined loading, empty, and error state — audit all pages, not just the ones built most recently.
- Check spacing, color, and typography consistency against the design tokens defined in Phase 3.A; fix any one-off deviations found.
- Verify touch-target sizes (buttons, nav items) are usable on a real mobile screen, not just visually present.

Technical Specification

Applies across the full `frontend/src/pages/` and `frontend/src/components/` tree — this is a cross-cutting QA pass, not a single new feature.

Acceptance Criteria

- ☐ No horizontal scroll or overlapping elements at any of the three tested breakpoints, on any page.
- ☐ Every page has a visibly correct loading/empty/error state when manually triggered.
- ☐ A short written note lists every inconsistency found and fixed.

Edge Cases & Notes

- Pay particular attention to the Result page's long text blocks (symptoms/prevention/management) and the History table on narrow screens, as these are the most likely to overflow.

PROMPT 164 — Add consistent empty/loading/error states across every data-driven page

Objective: Add consistent empty/loading/error states across every data-driven page.

Why This Matters: A final responsive/visual QA pass is what separates a working prototype from something that looks and feels like a real product — directly supporting the project's goal that the app feel personally owned and professionally built, not a rough student demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add consistent empty/loading/error states across every data-driven page.
- Test every page at three breakpoints: mobile (~375px), tablet (~768px), and desktop (~1280px+).
- Ensure every data-driven component (stats cards, tables, result sections) has a defined loading, empty, and error state — audit all pages, not just the ones built most recently.
- Check spacing, color, and typography consistency against the design tokens defined in Phase 3.A; fix any one-off deviations found.

- Verify touch-target sizes (buttons, nav items) are usable on a real mobile screen, not just visually present.

Technical Specification

Applies across the full `frontend/src/pages/` and `frontend/src/components/` tree — this is a cross-cutting QA pass, not a single new feature.

Acceptance Criteria

- ☐ No horizontal scroll or overlapping elements at any of the three tested breakpoints, on any page.
- ☐ Every page has a visibly correct loading/empty/error state when manually triggered.
- ☐ A short written note lists every inconsistency found and fixed.

Edge Cases & Notes

- Pay particular attention to the Result page's long text blocks (symptoms/prevention/management) and the History table on narrow screens, as these are the most likely to overflow.

PROMPT 165 — Do a final visual QA pass against the design system (spacing, colors, typography) and fix in...

Objective: Do a final visual QA pass against the design system (spacing, colors, typography) and fix inconsistencies.

Why This Matters: A final responsive/visual QA pass is what separates a working prototype from something that looks and feels like a real product — directly supporting the project's goal that the app feel personally owned and professionally built, not a rough student demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Do a final visual QA pass against the design system (spacing, colors, typography) and fix inconsistencies.
- Test every page at three breakpoints: mobile (~375px), tablet (~768px), and desktop (~1280px+).
- Ensure every data-driven component (stats cards, tables, result sections) has a defined loading, empty, and error state — audit all pages, not just the ones built most recently.
- Check spacing, color, and typography consistency against the design tokens defined in Phase 3.A; fix any one-off deviations found.
- Verify touch-target sizes (buttons, nav items) are usable on a real mobile screen, not just visually present.

Technical Specification

Applies across the full `frontend/src/pages/` and `frontend/src/components/` tree — this is a cross-cutting QA pass, not a single new feature.

Acceptance Criteria

- ☐ No horizontal scroll or overlapping elements at any of the three tested breakpoints, on any page.
- ☐ Every page has a visibly correct loading/empty/error state when manually triggered.
- ☐ A short written note lists every inconsistency found and fixed.

Edge Cases & Notes

- Pay particular attention to the Result page's long text blocks (symptoms/prevention/management) and the History table on narrow screens, as these are the most likely to overflow.
